Structure-Aware Cross-Dialect SQL Transfer for Constructing High-Quality Seeds in DBMS Fuzzing

Anonymous Author(s)

***Abstract*—DBMS fuzzing mutates initial seed queries into new ones to explore execution paths and trigger crash bugs, making the quality and diversity of the initial seeds critical. In practice, built-in test suites are typically used as initial seeds, but many DBMSs lack such suites, especially for emerging database engines. Recent studies have applied LLMs to end-to-end translate test queries across DBMSs. However, such translation may introduce syntax and semantic errors due to hallucination and limited dialect knowledge. Moreover, even with available seeds, current fuzzers struggle to effectively mutate them because of limited support for SQL grammar, often leading to the loss of important query semantics.**

In this paper, we propose MORPH, a test-case-level structure-aware cross-dialect SQL transfer framework designed for constructing high-quality initial seeds and enabling effective mutation in DBMS fuzzing. It follows a three-step workflow. First, MORPH builds an Abstract Query Template (AQT) with dialect-independent structure extraction, preserving statement order and cross-statement dependencies. Second, MORPH performs AQT-Guided Dialect Instantiation to construct a target seed sequence under target-dialect constraints. Finally, MORPH uses parser feedback to fix remaining incompatibilities, ensuring queries are fuzzer-parsable and mutable. We evaluate MORPH on four popular DBMSs: DuckDB, ClickHouse, MonetDB, and Virtuoso. Compared with the state-of-the-art seeds transforming approach SEDAR, MORPH covers 26% more branches, finds 16 more bugs in 24 hours, and achieves 100%–272% higher semantic correctness for transferred seeds. Moreover, MORPH uncovers 28 previously unknown bugs, including 20 confirmed.

***Index Terms*—DBMS Fuzzing, Initial Seeds, Cross-DBMS SQL Transfer**

I. INTRODUCTION

Database management systems (DBMSs) provide the essential framework for organizing and accessing data. Ensuring that a DBMS operates reliably is critical for maintaining data integrity and business continuity. However, the complex architectures and intricate implementations of modern DBMSs inevitably introduce potential bugs, which can lead to system crashes, data loss, or service interruptions [1], [2]. Timely detection of such bugs is therefore crucial for maintaining the reliability of these DBMSs.

In recent years, fuzzing has emerged as an effective technique for uncovering crashes and detecting hidden errors [3]–[6]. Many DBMS fuzzers have already discovered lots of bugs in popular DBMSs [7]–[9]. These fuzzers typically generate new test cases by repeatedly mutating a set of initial seed queries to explore more execution paths. Consequently, the effectiveness of fuzzing largely depends on the quality and diversity of the initial seeds [10]–[13]. While established DBMSs (e.g., PostgreSQL [14]) possess extensive test suites serving as ideal seeds, emerging engines like DuckDB [15] often lack such assets, creating a critical bottleneck that restricts the testing depth of existing fuzzers.

To overcome this limitation, existing studies and tools have used SQL dialect translation to transfer SQL across DBMSs [16]–[19]. General-purpose translators such as SQLGlot [20] and jOOQ [21], as well as recent systems such as RISE [22], mainly focus on statement-level translation for query migration or semantic-equivalent SELECT rewriting. However, fuzzing seeds are full test cases rather than isolated statements: they often contain multiple interdependent DDL, DML, and query statements, and the transferred results must remain parsable and mutable by downstream fuzzers. Thus, directly applying statement-level translators may lose cross-statement dependencies or produce fuzzer-incompatible structures. Some fuzzing-oriented methods instead aim to migrate high-quality test seeds from mature DBMSs to target systems, converting historical test cases into initial fuzzing seeds compatible across DBMS dialects [23], [24]. A typical example is SEDAR [19], which follows an LLM-centric *Translate-and-Sanitize* pipeline. It first executes existing test cases on the source DBMS to collect schema information, then directly feeds the schema and raw SQL text to an LLM for target-dialect translation. To ensure compatibility with fuzzers, SEDAR relies on *Unparsable Commenting*, which temporarily comments out unparsable parts during mutation.

Despite being able to transform SQL across DBMSs, existing dialect translation approaches cannot directly support fuzzing seed generation because they still suffer from *test-case semantic loss* and *fuzzer incompatibility*. (1) First, statement-level translators process statements in isolation, while fuzzing seeds rely on dependencies across schema definitions, inserted data, and subsequent queries. Losing these dependencies can break schema references, type constraints, or function semantics, making the transferred seeds fail during execution on the target DBMS. (2) Second, even when translation produces target-dialect SQL, the result may exceed the grammar accepted by downstream fuzzers. Existing methods often handle such incompatibilities by mechanically discarding or commenting out unsupported grammar structures, causing the loss of valuable SQL features and testing logic embedded in the original test cases. As a result, syntactically valid transferred queries can still suffer semantic degradation, limiting execution-path coverage and weakening optimizer stress testing.

To address the limitations of existing approaches, we propose a structure-aware method for cross-dialect transfer of SQL test cases. Our core idea is to **reformulate SQL transfer from direct SQL-text translation into AQT-Guided Dialect**

Instantiation for whole test cases. Instead of translating each SQL statement or raw test case as text, MORPH first extracts an AQT that explicitly captures schema dependencies, nested structures, and clause-level relationships. It then instantiates the AQT into target-dialect SQL using local AQT-node mappings and target-side feedback, with the LLM used only for bounded rule generation or local repair under structural constraints. Unlike statement-level translators such as RISE [22], which reduce an individual SQL statement to a statement-local rewrite rule, MORPH abstracts a whole test case into an AQT that explicitly records cross-statement read/write dependencies, an evolving schema context, and nested clause-level structure; mapping rules are only local operators for instantiating AQT nodes, rather than the core translation paradigm. This enables fuzzing to retain testing-relevant SQL features while avoiding the translation failures and destructive sanitization caused by direct text translation.

However, realizing this idea has three challenges: (1) First, query logic is often tightly coupled with dialect-specific syntax, requiring the separation of syntactic differences while preserving the original structure, dependencies, and semantic richness. (2) Second, ensuring target-side semantic correctness during query transformation is non-trivial, as operators, functions, and type conversions may differ across dialects, requiring carefully constrained rewriting. (3) Third, advanced SQL features in target DBMSs often exceed the grammar supported by generic fuzzers; naively removing unsupported constructs can break queries, necessitating structure-preserving adaptations into parsable forms without modifying the fuzzer.

To address these challenges, we propose MORPH, a structure-aware cross-dialect DBMS fuzzing framework for constructing high-quality initial seeds and enabling effective mutations. MORPH consists of three steps: (1) First, MORPH parses each SQL test case into schema-bound statement ASTs, replaces dialect-specific syntax with generic labels, and extracts both intra-statement structure and cross-statement dependencies to build an AQT. (2) Second, MORPH performs AQT-Guided Dialect Instantiation, recursively instantiating AQT nodes into dialect-specific subtrees under target-dialect constraints while propagating the evolving target schema context across statements. (3) Finally, MORPH adapts SQL queries that are incompatible with fuzzers by using runtime feedback, converting unsupported constructs into parsable forms while preserving their logical structure to maximize testing coverage.

We implement MORPH and evaluate it against the state-of-the-art approach SEDAR on four popular DBMSs, namely DuckDB, ClickHouse, MonetDB, and Virtuoso. The results show that MORPH achieves 100%–272% higher semantic correctness for transferred seeds than SEDAR. Based on that, MORPH finds a total of 7%–26% more branches and 1–11 more bugs than SEDAR on four DBMSs. More importantly, MORPH uncovers 28 previously unknown bugs, including 20 that have been confirmed, with 14 that have already been fixed. Moreover, MORPH has shown substantial real-world impact: all reported bugs of MonetDB have been incorporated into its official NEXTRELEASE milestone [25].

In summary, the paper makes the following contributions:

1) We find that many DBMS fuzzers lack high-quality initial seeds. LLM-based end-to-end translation often introduces errors, and limited SQL grammar support further hinders mutation and degrades query semantics, reducing testing effectiveness.
2) We propose MORPH, a structure-aware cross-dialect SQL transfer framework for constructing high-quality seeds. Instead of direct translation, MORPH extracts AQTs and performs AQT-Guided Dialect Instantiation with a retrieval-augmented knowledge base. Moreover, MORPH enables effective fuzzing through parser feedback-driven adaptation.
3) MORPH uncovered 28 previously unknown bugs on four diverse DBMSs, with 20 confirmed and 14 fixed. Compared to the state-of-the-art method SEDAR, MORPH improves semantic correctness by 100%–272%, achieving up to a 26% increase in branch coverage.

II. BACKGROUND AND MOTIVATION

SQL Dialects. *Database Management Systems (DBMSs)* underpin modern data-driven applications and expose high-level declarative query languages for data interaction. Although SQL is standardized, real-world DBMSs implement diverse *SQL dialects* with substantial syntactic and semantic variations. For example, PostgreSQL uses `AGE(timestamp)` for date arithmetic, whereas DuckDB requires `date_diff()`. Similarly, type casting syntax varies between `::INTERVAL` in PostgreSQL and `CAST(... AS INTERVAL)` in standard SQL. These dialect gaps pose substantial barriers for reusing test assets across different DBMSs. **DBMS Fuzzing and Cross-Dialect Seed Transfer.** *Fuzzing* has emerged as a core technique for discovering system crashes [9], [13]. By continuously generating and mutating inputs to traverse diverse execution paths, it effectively exposes corner cases and potential bugs. The efficacy of fuzzing is heavily influenced by the quality of *Initial Seeds*. Due to the diverse features of DBMSs, fuzzers lacking high-quality initial seeds with specific syntactic features (e.g., recursive CTEs) are often confined to shallow parsing logic, thus leaving many bugs undetected [26].

To address seed scarcity for emerging DBMSs testing, Cross-DBMS SQL Transfer has emerged as an active research direction [16]. Recent approaches leverage Large Language Models (LLMs) for cross-dialect translation [18], [27]. SEDAR [19], the current state-of-the-art, adopts a *Translate-and-Sanitize* paradigm: using an LLM to translate seeds between different dialects, followed by *Unparsable Commenting* to handle parsing errors by commenting out incompatible parts or statements.

Motivating Example. LLM-based cross-dialect transfer methods help in the generation of initial fuzzer seeds but may introduce *semantic deviation* due to hallucinations. Moreover, many fuzzers cover only a subset of SQL grammar. Existing methods typically comment out unsupported features to support seed mutation after transfer, which alters the query and causes the *logic loss*. *Limitations of Existing Approaches.* Figure 1 illustrates the semantic deviation and logic loss SEDAR experiences when transferring a test case from PostgreSQL

to DuckDB, which contains dialect-specific features like the `AGE` function in the `WHERE` clause and the `::INTERVAL` type cast. With limited knowledge of DuckDB's syntax, the LLM hallucinates and preserves `AGE`, causing syntax and semantic errors. To satisfy parsing constraints, SEDAR comments out `AGE` and replaces the `WHERE` condition with `True`, producing a syntactically valid but simplified query that loses the original verification intent and fails to exercise deep internal states. As a result, the migrated seed has limited utility for subsequent effective mutation in DBMS fuzzing. *Basic Idea of* MORPH. The core idea of MORPH is to thoroughly decouple logical intent from syntactic representation, shifting from unstable textual translation to more robust structural recursive substitution. As shown in Figure 1, MORPH does not perform erroneous syntactic translation, but instead abstracts the dialect-specific `AGE` function into a generic ¡TimeDiff¿ intent. It then performs a high-fidelity instantiation into DuckDB's equivalent syntax. By ensuring the type casting and window function partition logic are preserved, MORPH generates a high-fidelity seed that retains the original filtering complexity. By maintaining these complex semantics during transfer, MORPH ensures the generated seeds are effective for validation, enabling them to reach deeper code paths within the DBMS and uncover intricate logic bugs that might be unreachable by SEDAR.

III. APPROACH

MORPH is a structure-aware cross-dialect DBMS fuzzing framework for generating effective seeds for the target DBMSs. As Figure 2 shows, it takes source SQL test cases from mature DBMSs as input, transforms them across dialects into initial seed sequences for the target DBMS, and then mutates these seeds to discover bugs. Unlike LLM-centric text translation, MORPH treats transfer as an AQT-Guided Dialect Instantiation process. It first abstracts each source test case into an AQT that records statement order, schema dependencies, nested query structures, and clause-level relationships; then it recursively instantiates abstract nodes with rules retrieved or generated under target-dialect constraints. When a generated structure is rejected by the target parser or executor, MORPH performs feedback-driven local repair instead of destructively discarding the invalid structure, producing fuzzer-compatible seeds while retaining testing-relevant SQL features.

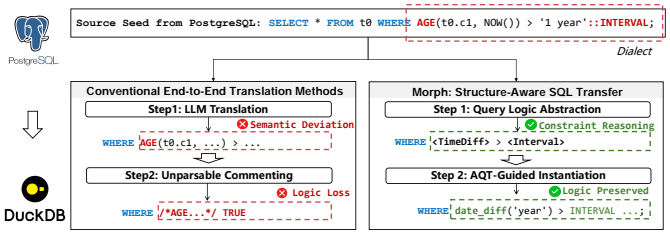

Fig. 1. Motivating example: SEDAR causes semantic deviation and logic loss, while MORPH preserves intent via AQT.

A. Structure-Preserving Abstraction

The step aims to separate the logic of test cases from their dialect-specific representations. The challenge lies in extracting such logic from tightly coupled SQL dialects. To address this, we formalize the seed structure to enable precise manipulation of the extracted logic. We first give some formal definitions of basic concepts. **Formal Definition.** To support syntax-aware query abstraction and decouple the logical intent of a test case, we introduce SQL abstraction rules and AQT. A SQL test case is an ordered sequence of DDL, DML, and query statements; R_i and W_i denote the objects read/referenced and written/created by statement s_i, respectively. The two formal objects are defined as follows:

Definition III.1 (SQL Abstraction Rule)**.** *Abstraction rules map from one dialect-specific construct to abstract representations. A rule has the form:*

$$[\mathit{Function} \mid \mathit{Type} \mid \cdots] \rightarrow [\langle \mathit{FuncTag} \rangle \mid \langle \mathit{TypeCast} \rangle \mid \cdots]$$

where Source is a dialect-specific SQL element and Target is its corresponding abstract label that represents the semantic intent. For example, $\texttt{AGE}(\cdot) \rightarrow \langle \mathit{TimeDiff} \rangle$ *abstracts interval computation.*

Definition III.2 (Abstract Query Template)**.** *Given a SQL test case* $\mathcal{C} = \langle s_1, \ldots, s_n \rangle$*, its AQT is defined as a 5-tuple:*

$$\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$$

- $\mathbf{U} = \langle \mathcal{U}_1, \ldots, \mathcal{U}_n \rangle$ *is the ordered sequence of statement templates. Each* $\mathcal{U}_i = \langle kind_i, S^i_{norm}, \mathcal{G}^i, R_i, W_i \rangle$ *records the statement type, normalized skeleton, intra-statement structure graph, read/reference set, and write/effect set of* s_i*.*
- $\Delta \subseteq \{(i,j) \mid i < j\}$ *is the cross-statement dependency set, where* $(i,j) \in \Delta$ *iff* $R_j \cap W_i \neq \emptyset$*.*
- $\mathcal{D}$ *is the cumulative schema context after executing* $\langle s_1, \ldots, s_n \rangle$ *in order, including table definitions, column types, constraints, and object-state summaries needed for target-side instantiation.*
- $\mathcal{G}_{tc} = (V_{tc}, E_{tc})$ *is the test-case dependency graph, where* $V_{tc} = \{1, \ldots, n\}$ *and* $E_{tc} = \Delta$*.*
- $\mathcal{H}$ *is a structural hash for deduplication, computed over the ordered normalized skeletons and dependency edges.*

Structure-Aware AQT Generation. Unlike prior statement-level translators that process each SQL statement in isolation and emit a statement-local rewrite rule, MORPH treats the whole test case as the abstraction unit, representing it as the AQT five-tuple of Definition III.2. Algorithm 1 scans the ordered statement sequence, extracts a statement template for each statement, records cross-statement dependencies, and propagates the evolving schema context across the whole test case. For each statement, EXTRACTSTMTTEMPLATE parses the statement with the current context, detects scoped SQL boundaries such as CTE bodies and nested subqueries, and applies abstraction rules $\mathcal{R}$ to replace dialect-specific constructs with generic representations. It also returns read/write sets (R_i, W_i), which

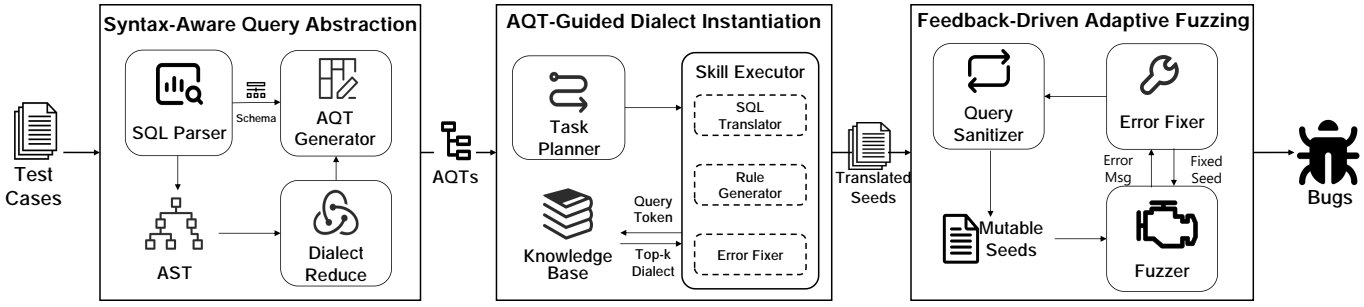

Fig. 2. **Overview of MORPH.** MORPH combines structure-preserving transfer with feedback-guided fuzzing.

are used to add dependency edges from earlier statements (Line 5). Finally, MORPH updates the cumulative context after each statement and assembles the AQT with the ordered statement units, dependency graph, cumulative schema context, and structural hash (Lines 7–11). This representation enables SQL test cases to be manipulated as structured graph-based objects rather than flat strings. For example, Figure 3 shows

Algorithm 1: AQT Extraction

Input : Test case $\mathcal{C} = \langle s_1, \ldots, s_n \rangle$; initial context $\mathcal{S}_0$; rules $\mathcal{R}$

Output : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$

1 $\mathbf{U} \leftarrow [\,]$; $\Delta \leftarrow \emptyset$; $Ctx \leftarrow \mathcal{S}_0$;
2 **foreach** s_i ***in*** $\mathcal{C}$ **do**
3 $\quad \mathcal{U}_i \leftarrow$ EXTRACTSTMTTEMPLATE$(s_i, Ctx, \mathcal{R})$;
4 $\quad (R_i, W_i) \leftarrow$ ACCESSSETS$(\mathcal{U}_i)$;
5 $\quad \Delta \leftarrow \Delta \cup \{(j, i) \mid j < i \wedge R_i \cap W_j \neq \emptyset\}$;
6 $\quad \mathbf{U}$.APPEND$(\mathcal{U}_i)$;
7 $\quad Ctx \leftarrow$ UPDATECONTEXT(Ctx, s_i) ; `// Evolve schema/state`
8 $\mathcal{D} \leftarrow Ctx$;
9 $\mathcal{G}_{tc} \leftarrow (\{1, \ldots, |\mathbf{U}|\}, \Delta)$;
10 $\mathcal{H} \leftarrow$ SHA256$(\bigoplus_i$ SKELETON$(\mathcal{U}_i) \oplus \Delta)$;
11 **return** $\langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$;

one statement from a source SQL test case containing a nested subquery, where the outer query filters records using an interval computation, e.g., "`SELECT ... WHERE u.created_at ¿ (SELECT AGE(...) FROM ...)`". The statement is first parsed into an AST, and structural boundaries are identified

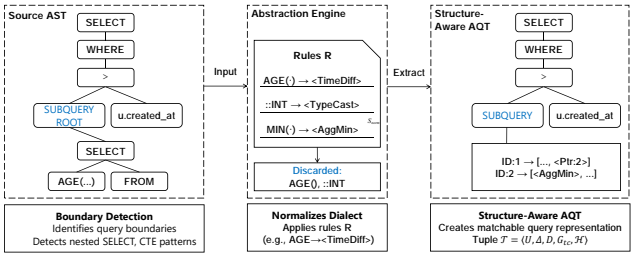

Fig. 3. An example of the per-statement component in AQT extraction. It detects nested SQL boundaries and normalizes dialect-specific features to abstract labels, producing the statement template used by the full test-case representation.

during recursive traversal. The `SUBQUERY` root node is recognized as a boundary, decomposing the query into an outer block (ID 1) and an inner subquery (ID 2). The abstraction engine then applies rules $\mathcal{R}$ to normalize dialect-specific constructs, mapping `AGE(...)`, `::INT`, and `MIN(.)` into ¡TimeDiff¿, ¡TypeCast¿, and ¡AggMin¿. Finally, a statement template is constructed, capturing the parent-child relationship via a pointer ¡Ptr:2¿ and producing a normalized skeleton S_{norm} that preserves logic while abstracting dialect differences.

B. AQT-Guided Dialect Instantiation

This step instantiates AQTs into syntactically valid target seed sequences while preserving semantic-rich structures from the source seed. The instantiation respects the dependency order in $\mathcal{G}_{tc}$: DDL and DML statements are instantiated before queries that depend on them, and the evolving target context is propagated across statements to maintain cross-statement type, reference, and state consistency. The challenge is preserving target-side semantic correctness during recursive AQT-node instantiation without breaking the original test-case order. To address it, MORPH transforms the instantiation task into a closed-loop reasoning process built on a Retrieval-Augmented Generation (RAG) architecture, which grounds LLM outputs in retrieved dialect-specific evidence to reduce hallucinations. We formulate dialect translation as a two-level instantiation problem: the outer level walks the statement dependency graph while preserving the original sequence for independent statements, and the inner level recursively instantiates each statement's structure graph in a bottom-up manner. To guarantee that the generated queries remain semantically consistent with the target dialect during recursive construction, the system employs an AQT-Guided Dialect Instantiation process as formalized in Algorithm 2. The algorithm first derives a stable dependency order over $\mathcal{G}_{tc}$, which respects cross-statement dependencies while preserving the source order for independent statements (Line 1). For each statement, it invokes INSTANTIATESTMT with source-side constraints and the current target context, and then propagates the updated target context to subsequent statements (Lines 4–6). Inside INSTANTIATESTMT, MORPH follows the intra-statement structure graph in bottom-up order, substitutes instantiated child fragments, retrieves or generates local dialect mappings from the Hybrid KB, and validates the resulting fragments

Algorithm 2: AQT Instantiation

Input : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathcal{D}, \mathcal{G}_{tc}, \mathcal{H} \rangle$; target dialect D_t; Hybrid KB HKB; threshold δ
Output : Compatible seed sequence $\mathcal{Q}_{target}$

$StmtOrder \leftarrow$ StableTopologicalSort$(\mathcal{G}_{tc}, \langle 1, \ldots, |\mathbf{U}| \rangle)$;
// Respect deps; keep original order
$\mathcal{Q}_{target} \leftarrow [\,]$; $C_s \leftarrow \mathcal{D}$; $C_t \leftarrow \emptyset$;
foreach i ***in*** $StmtOrder$ **do**
  $Q_i \leftarrow$ InstantiateStmt$(\mathcal{U}_i, D_t, HKB, \delta, C_s, C_t)$;
  $\mathcal{Q}_{target}$.Append(Q_i);
  $C_t \leftarrow$ UpdateContext(C_t, Q_i) ; // Propagate context
return $\mathcal{Q}_{target}$;

with the target DBMS parser before caching new rules. If validation fails, the Error Fixer repairs the fragment and rule using parser feedback; unverified candidates are discarded rather than reused. This mechanism avoids degenerate fixes such as clause removal and preserves testing-relevant SQL features. Finally, the instantiated statements form the target seed sequence $\mathcal{Q}_{target}$. **Test-Oriented Skill Execution.** The above instantiation procedure is implemented by *Skill Executor*, which acts as the reasoning core of the agent, with three specialized skills. *SQL Translator:* This skill is responsible for token-to-dialect instantiation. It generates the target SQL fragment by integrating the input segment with function signatures retrieved from the *Vector Index*. For example, when encountering `<TimeDiff>`, it maps the abstraction to the corresponding DuckDB `date_diff` signature to produce a dialect-compliant expression. *Rule Generator:* This skill is activated when retrieval confidence for a given feature falls below a predefined threshold. It uses the LLM's zero-shot reasoning, AST context, and few-shot examples to synthesize a candidate local rule, not a committed KB entry. The candidate is applied by the SQL Translator and, if needed, repaired by the Error Fixer; it is cached in the Hybrid KB only after target-parser validation succeeds. *Error Fix:* This skill operates as a runtime correction module. It performs immediate syntax validation on generated fragments, and when a target database parser rejects a fragment, it identifies and localizes the error using both the compiler feedback and the AST context. To prevent degenerate repairs such as clause removal, its correction process explicitly favors structure-preserving rewrites through constrained few-shot examples, ensuring that the final output remains a *Compatible Seed* while preserving the original test-case complexity. To reduce semantic drift, MORPH uses a Hybrid Knowledge Base with two complementary stores. The *Abstract Pattern Store* keeps deduplicated structural skeletons extracted during AQT generation (e.g., `JOIN`/`WINDOW` templates), preserving query topology across dialects. The *Vector Index* stores embedded dialect evidence (documentation, function signatures, and operator semantics) and retrieves top-k relevant constraints for each abstract token τ, thereby grounding generation in concrete dialect evidence rather than free-form synthesis. Take Figure 4 as an example, the left panel shows the abstract query graph $\mathcal{G}$, where the root segment id_1 depends on the nested segment id_2; therefore, the planner first performs a topological sort and obtains the execution order $Order = [id_2 \rightarrow id_1]$. In the center panel, MORPH first instantiates id_2 by concretizing its abstract tokens, including mapping `<AggMin>` to `MIN(..)` and `<TypeCast>` to a dialect-valid `CAST(.. AS ..)` form, and validates this subquery fragment with parser feedback until it succeeds. It then substitutes this validated id_2 fragment into id_1 and instantiates the outer predicate by mapping `<TimeDiff>` to DuckDB's `date_diff`. The right panel shows the resulting *Compatible Seed* that keeps the original nested time-window filtering logic.

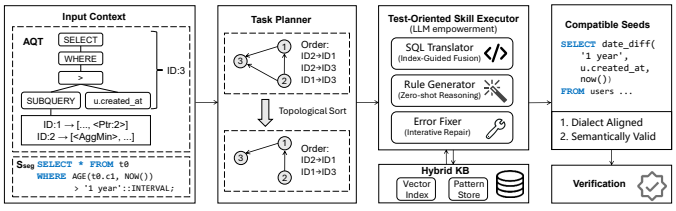

Fig. 4. RAG-enhanced AQT-Guided Dialect Instantiation.

C. Feedback-Driven Adaptive Fuzzing

Some DBMS features lie beyond the grammatical scope supported by existing fuzzers, making mutation difficult. To enable mutation over such seeds, the challenge is to translate or adapt advanced DBMS features into fuzzer-compatible forms, or to mark mutable regions, while preserving both syntax correctness and semantic consistency. To address that, MORPH iteratively leverages parsing and execution feedback to rewrite or encapsulate unsupported constructs into valid equivalents through a structure-preserving fallback process. As Figure 5 shows, upon receiving compatible seeds, MORPH sanitizes and executes them in the target DBMS, using execution results to either proceed to mutation or trigger error-guided semantic repair, and produces a seed annotated with mutable regions to support fuzzing. Specifically, when a transferred seed Q_{target}

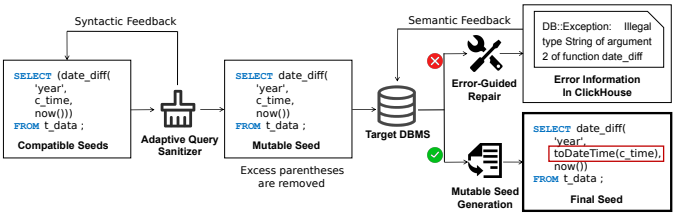

Fig. 5. Feedback-driven adaptive fuzzing workflow.

is fed into the fuzzer, it may still contain subtle syntax incompatibilities not captured by the static translation phase. Instead of discarding these seeds or destructively commenting out unparsable tokens (as in SEDAR [19]), the sanitizer captures the parser error message, localizes the unsupported dialect-specific construct to the corresponding AQT node, and repairs it using syntactic feedback. For instance, in Figure 5, it

automatically removes excess brackets around the `date_diff` function to produce a syntactically valid Mutable Seed. When an error signal is received (e.g., "Syntax error near SELECT"), MORPH applies four fallbacks in order. First, it checks the sanitization dictionary and applies an existing mapping when the unsupported construct has a known target-compatible form. Second, if no exact mapping exists, it uses RAG to retrieve candidate replacements from dialect evidence and validates the rewritten fragment with the fuzzer parser. Third, if no suitable replacement can be found, MORPH executes the original source-side construct in the source DBMS and substitutes the returned value as a constant, preserving the surrounding query structure whenever possible. Finally, only if these structure-preserving attempts fail, MORPH comments out the minimal offending construct. This ordering makes destructive commenting a last resort rather than the default sanitization strategy. Once a seed passes the fuzzer's parser, it is designated as a *Mutable Seed*. MORPH then identifies data-dependent components, such as literals and predicates, and marks them as mutable regions. This directs the fuzzer's mutation engine to focus on logic-altering variations rather than the fixed syntactic skeleton, accelerating the discovery of deep logic bugs.

IV. EVALUATION

MORPH is implemented in Python and integrates GRIFFIN [10], a coverage-guided mutation fuzzer, as the downstream execution engine. For the LLM backbone powering the Dynamic Skill Executor and Rule Generator, we employ Gemini 2.5-flash-lite [28] as the primary language model due to its strong code understanding capabilities and cost efficiency. We set the temperature parameter to 0.2 to balance determinism and output diversity, as justified by the sensitivity analysis discussion in Section V. We evaluate MORPH to assess its performance in cross-dialect seed transformation and its practical impact on DBMS fuzzing effectiveness. Specifically, our evaluation focuses on the following research questions:

- **RQ1:** Can MORPH find previously unknown bugs in DBMSs?
- **RQ2:** How does MORPH compare to other fuzzing tools?
- **RQ3:** How does MORPH compare to other translation methods in terms of seed validity and semantic richness?
- **RQ4:** What is the contribution of each component in MORPH to the overall performance?

We first describe the experimental setup, and then answer RQ1–RQ4 in the following four subsections.

A. Experimental Setup

Target DBMSs and Environment. To evaluate the generalization of MORPH across diverse architectures, we selected four widely used DBMSs according to the DB-Engines Ranking [29]: MonetDB [30] v11.51, Virtuoso [31] v7.2, DuckDB [15] v1.1.3, and ClickHouse [32] v24.8 LTS. These systems span a broad spectrum of architectures, from embedded OLAP engines to distributed columnar stores, and while they share the relational data model, each implements a distinct SQL dialect. We explicitly excluded traditional row-store OLTP systems (e.g., MySQL and PostgreSQL) from the target list, as they serve as the source DBMSs for seed collection. Instead, we prioritized OLAP and multi-model architectures to rigorously evaluate MORPH's capability in bridging significant dialectal and architectural gaps (i.e., from Row-store like MySQL to Column-store like DuckDB). All experiments were conducted on a Linux server equipped with an Intel Xeon CPU and 256 GB of RAM. To ensure reproducibility and isolation, each DBMS instance ran within a separate Docker container.

TABLE I
STATISTICS OF COLLECTED OPEN-SOURCE TEST CASES FROM SOURCE DBMSS.

Source DBMS	Test Cases	Statements	Size
SQLite	2,643	308,292	31 MiB
MySQL	2,685	98,862	15 MiB
PostgreSQL	1,367	19,814	8.4 MiB
Total	6,695	426,968	54.4 MiB

Collected Test Cases. Following the prior methodology [19], we collected built-in test cases from mature DBMS repositories (i.e., SQLite [33], MySQL [34], and PostgreSQL [35]) for transfer. Table I shows the statistics on collected test cases in each DBMS. Specifically, the dataset aggregates 426,968 SQL statements across 6,695 files, totaling 54.4 MiB of query text.

Baselines and Evaluation Metrics. For translation effectiveness, we compare MORPH against six baselines, including No translation, SQLGlot [20], jOOQ [21], CrackSQL [36], RISE [22], and SEDAR [19]. The No translation strategy directly executes the original test cases collected from source DBMSs without modification. SQLGlot and jOOQ represent off-the-shelf SQL dialect translators, while CrackSQL and RISE represent recent SQL translation systems. SEDAR represents the current state-of-the-art LLM-based transfer tool in constructing initial seeds in fuzzing. We reproduced SEDAR's complete pipeline, including schema capture, end-to-end LLM translation, and its comment-out sanitization strategy.

We employ three metrics to evaluate MORPH. First, to assess transfer quality, we measure *Syntactic Correctness* (the proportion of seeds passing the parser) and *Semantic Correctness* (the proportion executing without runtime errors). Formally, given a generated seed set $\mathcal{Q}$, we compute $\text{Syn.} = |\mathcal{Q}_{parse}|/|\mathcal{Q}|$ and $\text{Sem.} = |\mathcal{Q}_{exec}|/|\mathcal{Q}|$, where $\mathcal{Q}_{parse}$ denotes parser-accepted seeds and $\mathcal{Q}_{exec}$ denotes seeds executed without runtime errors. Here, semantic correctness does not denote result equivalence between the source and target DBMSs; instead, it captures whether a transferred seed is executable in the target DBMS without runtime errors. We separately measure semantic richness by counting valid functions, data types, and keywords preserved in executable transferred seeds. Second, to quantify the effectiveness in improving fuzzing, we utilize LLVM's `SanitizerCoverage` to track *Branch Coverage* over 24-hour campaigns. We report the average results across five independent runs to mitigate randomness. Third, regarding real-world bug discovery, we present the detected number of

bugs. The bugs are deduplicated from raw crash logs using stack hashes and error signatures, ensuring that the reported numbers reflect unique bugs. We report bug counts under two complementary scopes. RQ1 reports the full real-world bug-discovery campaign of MORPH, including all unique previously unknown bugs we detected and their confirmation/fix status. RQ2 reports only the bugs found under the controlled 24-hour baseline-comparison setting, where all tools run with the same time budget and deduplication procedure. Thus, the 28 detected bugs in RQ1 measure practical impact, whereas the 22 bugs in RQ2 measure fair 24-hour comparative effectiveness.

B. RQ1: Real-World Bug Detection

We deployed MORPH with the backend fuzzer GRIFFIN [10] on the latest development versions of the four target DBMSs. This RQ reports the full real-world bug-discovery campaign, not the restricted 24-hour baseline-comparison setting used in RQ2. As summarized in Table II, MORPH uncovered 28 unique bugs. Among these issues, 20 have been confirmed and are pending patches, and 14 critical bugs have already been fixed by the vendors. For instance, regarding the fixed bug in DuckDB (#205xx), the developers appreciated the report exposing an intricate type inference flaw in the *Executor*, which was triggered by preserving complex cross-table correlations from PostgreSQL. They merged a fix within days, further validating that MORPH successfully identified deep-seated bugs missed by existing test suites. Moreover, all reported bugs of MonetDB have been incorporated into its official NEXTRELEASE milestone [25]. The detected bugs are not limited to the surface level (e.g., Parser) but penetrate deep into the DBMS kernels. As shown in the *Component* column of Table II, the bugs span critical subsystems including the Optimizer, Binder, and Storage Engine.

Case Study: An internal error in DuckDB. To illustrate the effectiveness of MORPH, we examine a fixed bug in DuckDB (#205xx) triggered by a complex query involving a nested `CROSS JOIN` and specific DDL configurations. Generally, in SQL dialects, join operations and subqueries are standard; however, the structural interaction between specific join types and aliased subqueries can induce a crash bug in DuckDB when executing the statement shown in the PoC. In this case, the source logic was derived from a PostgreSQL test case featuring a `CREATE TABLE` statement with `DOUBLE PRECISION` and a query containing a nested `CROSS JOIN`.

TABLE II
STATISTICS OF PREVIOUSLY UNKNOWN BUGS IN POPULAR DBMSS DETECTED BY MORPH. [**UAF**: USE-AFTER-FREE, **AF**: ASSERTION FAILURE, **NPD**: NULL POINTER DEREFERENCE, **DBZ**: DIVIDE-BY-ZERO, **SEGV**: SEGMENTATION VIOLATION, **OOM**: OUT OF MEMORY]

Confirmed/Detected	Component	Bug Type
MonetDB (15/17)	optimizer server function	SEGV(6), NPD(1), AF(1) SEGV(4), DBZ(1), AF(2) SEGV(1), NPD(1)
Virtuoso (2/5)	server	SEGV(4), UAF(1)
DuckDB (2/4)	executor binder function	AF(1), DBZ(1) SEGV(1) OOM(1)
ClickHouse (1/2)	function server	SEGV(1) AF(1)
Total	7 components	**20 confirmed, 28 detected**

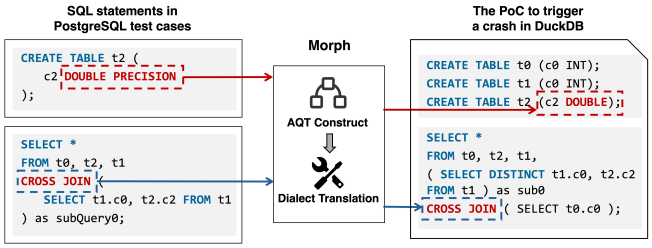

Fig. 6. An internal error in DuckDB(Bug #205xx) triggered by a transferred PostgreSQL test case.

MORPH's effectiveness stems from its structure-aware transfer. Take the case as an example, in PostgreSQL, nested joins involving a table with `DOUBLE PRECISION` and a subquery are well tested, whereas DuckDB lacks sufficient coverage for such combinations. Conventional approaches often simplify schemas (e.g., converting `DOUBLE` to `INT`) or flatten nested queries to ensure compatibility, sacrificing semantic fidelity. In contrast, MORPH preserves the full structural complexity of the original seed, forcing DuckDB to perform precise type inference and vector allocation for correlated joins and thereby exercising execution paths that simplified queries would not reach. This high-fidelity transfer exposes a critical flaw in the executor logic that remains undetected when structural complexity is removed.

Answer to RQ1: In the full real-world bug-discovery campaign, MORPH successfully exposed 28 unknown bugs in four popular DBMSs, affecting core components including the Optimizer and Storage Engine. Among them, 20 bugs have been confirmed, and 14 bugs have already been fixed by DBMS vendors.

C. RQ2: Comparison with Other Fuzzing Tools

We compared MORPH against three state-of-the-art DBMS testing tools: SQLancer [37] (logic/crash baseline [10], [11], [13]), SQUIRREL [13] (default built-in seed corpus [38]), and SEDAR [19]. All tools ran for 24 hours under identical settings, so the bug count in this RQ is a controlled comparison rather than the full real-world count in RQ1. We normalized coverage through a unified dry run and deduplicated crashes using stack-trace hashing plus manual inspection.

As illustrated in Table III, MORPH achieves the highest branch coverage across all evaluated DBMSs. Specifically, MORPH covers a total of 422,130 branches, representing overall improvements of 528%, 60%, and 14% over SQLancer, SQUIRREL, and SEDAR, respectively, on their supported targets.

TABLE III
NUMBER OF BRANCHES COVERED BY DIFFERENT TOOLS (24H).

DBMS	SQLancer	SQUIRREL	SEDAR	MORPH
MonetDB	-	43,747	76,142	**81,486**
Virtuoso	-	26,302	36,583	**40,183**
DuckDB	22,188	96,434	114,876	**144,822**
ClickHouse	25,618	-	141,215	**155,639**
Total	47,806	166,483	368,816	422,130
Improvement*	528%↑	60%↑	14%↑	-

* Calculated only for commonly supported DBMSs.

TABLE IV
NUMBER OF BUGS DETECTED BY DIFFERENT TOOLS (24H).

DBMS	SQLancer	SQUIRREL	SEDAR	MORPH
MonetDB	-	0	3	**14**
Virtuoso	-	0	2	**3**
DuckDB	0	0	1	**3**
ClickHouse	0	-	0	**2**
Total	0	0	6	22
Increment	22↑	22↑	16↑	-

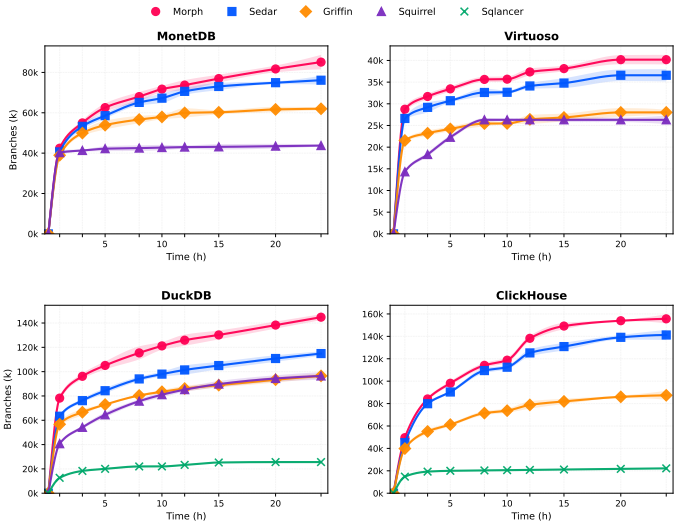

Fig. 7. Coverage growth over time in 24-hour fuzzing campaigns. Griffin denotes the downstream fuzzer with its original seed corpus.

Figure 7 further shows that MORPH reaches higher coverage earlier and continues to gain branches after 12 hours, while several baselines plateau sooner. The coverage improvement also brings more detected bugs. As shown in Table IV, MORPH exposes significantly more unique bugs than baselines. Moreover, MORPH detects a total of 16 more bugs than SEDAR on all four DBMSs. This proves that coverage gains are not merely statistical but functionally impactful, enabling the detection of bugs that remain unreachable to baselines. Unlike SEDAR, which sacrifices semantic depth through destructive sanitization, MORPH employs structure-aware transfer to retain the full logical complexity of source test cases. By ensuring high semantic correctness (RQ3), MORPH enables fuzzers to bypass early engine rejections, exercise deeper optimizer and executor logic, and trigger critical vulnerabilities.

TABLE V
SYNTACTIC AND SEMANTIC CORRECTNESS OF TRANSFERRED SEEDS.

Method	MonetDB Syn.	MonetDB Sem.	Virtuoso Syn.	Virtuoso Sem.	DuckDB Syn.	DuckDB Sem.	ClickHouse Syn.	ClickHouse Sem.
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
SEDAR	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
MORPH	**0.929**	**0.776**	**0.670**	**0.629**	**0.939**	**0.748**	**0.895**	**0.694**
vs. SEDAR	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./Sem.: syntactic/semantic correctness; "-": unsupported dialects.

Answer to RQ2: Under the controlled 24-hour comparison, MORPH covers 528%, 60%, and 14% more branches, and discovers 22, 22, and 16 more bugs than SQLancer, SQUIRREL, and SEDAR, respectively. The results demonstrate the effectiveness of MORPH in enhancing DBMS fuzzing.

D. RQ3: Effectiveness of Translation of MORPH

We evaluate MORPH against untranslated seeds, SQL dialect translators, recent SQL translation systems, and SEDAR from two aspects: transferred-seed validity and SQL-feature richness. This evaluation targets fuzzing seed quality rather than cross-DBMS result equivalence: valid seeds must execute on the target DBMS, and rich seeds should retain diverse SQL features for mutation.

Validity of the Translated Test Cases. As shown in Table V, directly executing source seeds yields low semantic correctness, dropping to 0.097 on Virtuoso and reaching only 0.284 even in the best case. Existing translators improve syntactic acceptance but still leave a semantic gap; on DuckDB, SQLGlot, CrackSQL, and RISE reach at least 0.738 syntactic correctness, yet no more than 0.314 semantic correctness. RISE's statement-level design is especially limited for seed transfer: in SQLite-to-DuckDB, MORPH improves semantic correctness from 0.314 to 0.776. When the translated outputs are used as initial seeds for fuzzing, MORPH further achieves 34% higher branch coverage than RISE and detects three bugs in DuckDB, whereas RISE triggers none. Compared with SEDAR, MORPH achieves the best result in every column, with 5%–19% higher syntactic correctness and 100%–272% higher semantic correctness across the tested DBMSs.

These gains confirm that AQT-Guided Dialect Instantiation maps source intent into target-dialect constraints while preserving parsability and executability.

Semantic Richness of the Translated Test Cases. Beyond validity, we measure whether transferred seeds retain testing value by counting valid functions, data types, and keywords. As shown in Table VI, MORPH preserves 1,036 functions, 180

TABLE VI
VALID SQL FEATURES IN TRANSFERRED SEEDS.

Method	MonetDB			Virtuoso			DuckDB			ClickHouse		
	F	DT	K	F	DT	K	F	DT	K	F	DT	K
No translation	107	**29**	235	28	16	102	185	34	291	83	36	124
SQLGlot	-	-	-	-	-	-	195	28	288	103	35	142
jOOQ	-	-	-	-	-	-	187	22	253	94	34	104
CrackSQL	-	-	-	-	-	-	198	32	329	146	46	136
RISE	145	27	236	45	18	122	251	34	314	155	54	160
SEDAR	132	**29**	244	52	17	136	214	39	340	227	61	152
MORPH	**161**	**29**	**370**	**87**	**24**	**335**	**456**	**63**	**476**	**332**	**64**	**187**
vs. SEDAR (%)	+22	+0	+52	+67	+41	+146	+113	+62	+40	+46	+5	+23

F/DT/K: functions/data types/keywords; "-": unsupported dialects.

data types, and 1,368 keywords, exceeding SEDAR by 411, 34, and 496, respectively (66%, 23%, and 57%).

The diversity gap mainly comes from sanitization strategy: SEDAR may comment out unparsable casts or function calls, whereas MORPH constructively instantiates abstract structures into target-compliant forms, preserving more feature-rich inputs for downstream mutation.

Answer to RQ3: MORPH achieves the strongest validity and feature-richness results, improving semantic correctness by up to 272% over SEDAR and preserving 411 more functions, 34 more data types, and 496 more keywords.

E. RQ4: Contribution of Individual Components

To analyze the specific contributions of the individual components within MORPH, we conducted an ablation study across two phases. For the *translation phase*, we evaluated four variants using the task of PostgreSQL-to-DuckDB transfer: (1) MORPH$_{base}$: the baseline using direct zero-shot LLM translation (i.e., prompting the LLM with the raw SQL and asking for a direct translation, unlike the S^- baseline which executes the raw, untranslated SQL directly); (2) MORPH$_{abs}$: equipped solely with *Syntax-Aware Query Abstraction*; (3) MORPH$_{rag}$: equipped solely with *AQT-Guided Dialect Instantiation*; (4) MORPH$_{full}$: the complete framework integrating both components.

For the *fuzzing phase*, since the *Feedback-Driven Adaptive Sanitizer* operates independently during runtime fuzzing and does not affect translation quality, we further introduce MORPH$_{static}$ as a controlled variant where the runtime feedback loop is disabled and invalid seeds are directly discarded, and compare it against MORPH$_{full}$ where the sanitizer actively repairs parsing errors to maximize seed acceptance.

Contribution of Syntax-Aware Query Abstraction and AQT-Guided Dialect Instantiation. As shown in Figure 8 (a) and (b), when both the *Syntax-Aware Query Abstraction* and the *AQT-Guided Dialect Instantiation* are disabled, the syntactic correctness is 51% and the semantic correctness of MORPH$_{base}$ is 27%. With *Syntax-Aware Query Abstraction*, MORPH$_{abs}$ improves syntactic correctness to 86% (+35%) by leveraging AQT to provide a more stable structural skeleton. The semantic correctness also increases to 32%. Moreover, with *AQT-Guided Dialect Instantiation*, MORPH$_{rag}$ further enhances syntactic correctness to 87%, and increases semantic correctness to 49% (+21%) by grounding generation in retrieved dialect evidence. The Full framework MORPH$_{full}$ achieves the optimal balance, reaching **95%** syntactic correctness and **65%** semantic correctness. Figure 8(c) and (d) show that MORPH$_{full}$ incurs more repair iterations and token overhead, but delivers the best trade-off between generation quality and efficiency.

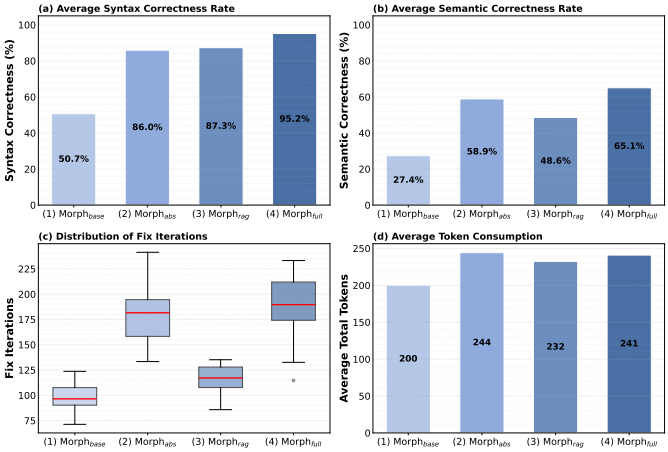

Fig. 8. Ablation analysis of MORPH components.

TABLE VII
SANITIZER ABLATION ON BRANCH COVERAGE (24H).

DBMS	MORPH$_{direct}$	MORPH$_{full}$	Improvement
MonetDB	67,620	**81,486**	**+21%**
Virtuoso	37,213	**40,183**	**+8%**
DuckDB	118,453	**144,822**	**+22%**
ClickHouse	112,083	**155,639**	**+39%**

Contribution of Feedback-Driven Adaptive Sanitizer. To evaluate the component, we compare MORPH$_{full}$ with MORPH$_{direct}$, which disables runtime feedback and discards seeds that fail parsing. Table VII presents MORPH covers 8% to 39% more branches across four DBMSs than MORPH$_{direct}$ over a 24-hour campaign. Without adaptive repair, MORPH$_{direct}$ suffers high parser-stage rejection. MORPH$_{full}$ repairs such errors, recovers invalid seeds, and lets fuzzers explore deeper target states.

Answer to RQ4: All components are critical: MORPH$_{abs}$ improves syntactic correctness by 37%, MORPH$_{rag}$ boosts semantic correctness by 21%, and MORPH$_{full}$ improves syntax/semantic correctness by 44%/38%. The sanitizer further increases branch coverage by up to 39%.

V. DISCUSSION

LLM Parameter Selection. We use T=0.2 because it achieves 95% translation success and 91% first-attempt stability, outperforming T=0.5 (92%/78%) and T=0.8 (85%/70%).

Impact of the Underlying LLM. Table VIII supports Gemini 2.5-flash-lite: Qwen3.5-27B is more semantically correct but costs 36× more and takes 9.5× longer. With

TABLE VIII
LLM COMPARISON. TS DENOTES TEST SUITE.

Model	Syn.	Sem.	Cost/TS	Time/TS
Gemini 2.5-Flash-Lite	0.929	0.776	0.003	146.80
GPT-3.5-Turbo	0.890	0.533	0.015	208.66
DeepSeek-V3	0.986	0.699	0.003	231.11
Qwen3.5-27B*	0.971	0.852	0.119	1395.6

* Qwen3.5-27B with depth thinking mode

TABLE IX
MANUAL EFFORT FOR EXTENDING MORPH TO TARGET DBMSS.

Target DBMS	Auto Rules	Manual	Ratio
DuckDB	534	42	7.3%
ClickHouse	1,362	46	3.3%
MonetDB	398	26	6.1%
Virtuoso	782	22	2.7%
Total	3,076	136	4.2%

GPT-3.5-Turbo, MORPH still outperforms SEDAR (0.890/0.533 vs. 0.781/0.388), showing that gains mainly come from the method. **Extensibility and Manual Effort.** To extend MORPH to another DBMS, we first automatically extract lexical and syntactic features from grammar specifications and register the corresponding mapping rules into the Hybrid KB. For features that cannot be mapped automatically, MORPH uses the LLM to generate candidate rules, followed by manual review and correction. Table IX summarizes the extension effort across four target DBMSs. Overall, only 4.2% (136/3,212) of transfer rules require manual correction, and each correction typically takes 1–3 minutes for an engineer with DBMS expertise.

These results show that most dialect support is obtained automatically, while manual review is limited to a small number of unmapped or ambiguous constructs. To quantify the benefit of this manual effort, we remove the 42 manually corrected DuckDB rules and rerun MORPH on DuckDB. The manually reviewed rules allow MORPH to cover 14,087 additional branches and find two more bugs than the fully automated version. Thus, manual review provides additional gains, but the majority of coverage and bug-finding capability comes from the automated extraction, instantiation, and repair pipeline.

VI. RELATED WORK

Related work falls into three categories: DBMS fuzzing, LLM-based seed generation, and cross-dialect SQL translation. The first category improves how DBMS inputs are generated, mutated, or checked; the second uses LLMs to synthesize or transfer testing inputs; and the third translates SQL across dialects for migration or compatibility. MORPH builds on these lines of work but targets a different problem: structure-preserving transfer of whole SQL test cases into high-quality fuzzing seeds.

A. DBMS Fuzzing

Fuzzing is one of the most effective automated testing techniques for identifying DBMS vulnerabilities. Generation-based tools, such as SQLsmith [9], SQLancer [37], [39], and DynSQL [12], construct SQL queries from hard-coded rules or semantic oracles, while recent clause-guided and differential methods further improve logic-bug detection [40]–[44]. However, predefined templates can limit complex scenarios [45]. Mutation-based fuzzers instead evolve seed pools: SQUIRREL [13] uses IR-based mutations, GRIFFIN [10] uses grammar-free metadata replacement, and LEGO [11] combines SQL statement sequences. These fuzzers are effective once suitable seeds or generation rules are available, but they do not directly address how to obtain high-quality seeds for emerging DBMSs. Unlike these works, MORPH focuses on the upstream *seed acquisition* problem by transferring mature DBMS test cases into target-compatible seed sequences.

B. LLM-Based Seed Generation

Recent work has explored LLMs for generating or enriching testing inputs. Fuzz4All [17] uses LLMs as a general input generator, and ShQveL [46] synthesizes SQL fragments for database testing. SEDAR [19] is the closest work to MORPH because it also transfers existing DBMS test cases into fuzzing seeds. However, these methods largely rely on direct text generation or post-generation filtering and sanitization. In particular, SEDAR feeds schema information and raw SQL to an LLM, then repairs incompatibilities by sanitizing unparsable fragments. In contrast, MORPH confines LLM use to AQT-guided instantiation, where explicit schema dependencies, nested structures, and clause-level relationships support local repair without discarding context.

C. Cross-Dialect SQL Translation

General-purpose SQL dialect translators, such as SQLGlot [20] and jOOQ [21], convert SQL syntax across DBMS dialects for query migration and application compatibility. Recent research systems further improve SQL translation through learned or rule-driven mechanisms: CrackSQL [36] studies LLM-based dialect translation, RISE [22] performs rule-driven SELECT translation via query reduction, and QTRAN [47] extends metamorphic-oracle based testing across DBMS dialects. These systems provide useful statement-level mappings, but their goal is query migration, semantic-equivalent translation, or oracle portability rather than fuzzing-seed construction. They therefore do not explicitly model a seed as an ordered test case with DDL/DML dependencies, evolving schema context, and downstream fuzzer compatibility constraints. In particular, although the AQT may appear to resemble the query-reduction ("skeleton") step of RISE [22], the two constructs differ in both scope and representation. In scope, RISE operates on a single SELECT statement, whereas an AQT spans an ordered, multi-statement test case and therefore captures cross-statement read/write dependencies Δ and an evolving schema context $\mathcal{D}$ that do not arise within a single statement. In representation, RISE encodes a dialect as a statement-local AST rewrite rule $\langle Pat_{src}, Pat_{tar} \rangle$, whereas MORPH treats such token-level mappings as local operators that instantiate nodes of the AQT five-tuple (Definition III.2); the

dialect abstraction the two share is thus only the per-statement field S^i_{norm} of the AQT, not the representation itself. MORPH differs by treating cross-dialect transfer as test-case-level AQT instantiation: it preserves cross-statement dependencies while constructing target-compliant seed sequences for mutation-based DBMS fuzzing.

VII. CONCLUSION

We propose MORPH, an automated fuzzing framework that extracts dialect-agnostic AQTs, instantiates them through AQT-Guided Dialect Instantiation, and applies adaptive sanitization for cross-dialect DBMS fuzzing. Across four DBMSs, MORPH improves semantic correctness by up to 272%, achieves higher coverage than SEDAR, and detects 28 unknown bugs. Future work will extend this transfer paradigm to non-relational systems.

VIII. DATA AVAILABILITY

Artifacts and data are available in an anonymous Zenodo archive [48].

REFERENCES

[1] T. P. G. D. Group, "pgsql-bugs," https://www.postgresql.org/list/pgsql-bugs/, 2025, accessed: June 25, 2026.

[2] MySQL, "Mysql bug home," https://bugs.mysql.com/, 2025, accessed: June 25, 2026.

[3] M. Zalewski, "american fuzzy lop," http://lcamtuf.coredump.cx/afl/, accessed: June 25, 2026.

[4] "Libfuzzer," https://www.llvm.org/docs/LibFuzzer.html, accessed: June 25, 2026.

[5] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "Afl++ : Combining incremental steps of fuzzing research," in *USENIX Workshop on Offensive Technologies (WOOT)*, 2020.

[6] J. Liang, Y. Jiang, Y. Chen, M. Wang, C. Zhou, and J. Sun, "Pafl: Extend fuzzing optimizations of single mode to industrial parallel mode," 2018.

[7] "Sqlsmith score list," https://github.com/anse1/sqlsmith/wiki#score-list, accessed: June 25, 2026.

[8] "Bugs found in database management systems," https://www.manuelrigger.at/dbms-bugs, accessed: June 25, 2026.

[9] SQLsmith Contributors, "Sqlsmith," 2015, accessed: 2026-01-26. [Online]. Available: https://github.com/anse1/sqlsmith

[10] J. Fu, J. Liang, Z. Wu, M. Wang, and Y. Jiang, "Griffin: Grammar-free dbms fuzzing," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.

[11] J. Liang, Y. Chen, Z. Wu, J. Fu, M. Wang, Y. Jiang, X. Huang, T. Chen, J. Wang, and J. Li, "Sequence-oriented dbms fuzzing," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023, pp. 668–681.

[12] Z.-M. Jiang, J.-J. Bai, and Z. Su, "{DynSQL}: Stateful fuzzing for database management systems with complex and valid {SQL} query generation," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 4949–4965.

[13] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, "Squirrel: Testing database management systems with language validity and coverage feedback," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 955–970.

[14] "Postgresql github," https://github.com/postgres/postgres, accessed: June 25, 2026.

[15] "Duckdb github," https://github.com/duckdb/duckdb, accessed: June 25, 2026.

[16] X. Zhou, C. Chai, G. Li, and J. Sun, "Database meets artificial intelligence: A survey," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 3, pp. 1096–1116, 2020.

[17] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.

[18] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.

[19] J. Fu, J. Liang, Z. Wu, and Y. Jiang, "Sedar: Obtaining high-quality seeds for dbms fuzzing via cross-dbms sql transfer," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–12.

[20] SQLGlot Contributors, "SQLGlot: Python sql parser and transpiler," 2026, accessed: 2026-06-20. [Online]. Available: https://github.com/tobymao/sqlglot

[21] jOOQ Contributors, "jOOQ: Type safe sql in java," 2026, accessed: 2026-06-20. [Online]. Available: https://www.jooq.org/

[22] X. Xie, Y. Zhang, W. Dou, Y. Gao, Z. Cui, J. Song, R. Yang, and J. Wei, "Rise: Rule-driven sql dialect translation via query reduction," *arXiv preprint arXiv:2601.05579*, 2026.

[23] N. Mündler, J. He, H. Wang, K. Sen, D. Song, and M. Vechev, "Type-constrained code generation with language models," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 601–626, 2025.

[24] L. Lin, Z. Hao, C. Wang, Z. Wang, R. Wu, and G. Fan, "Sqless: Dialect-agnostic sql query simplification," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 743–754.

[25] "MonetDB NextRelease milestone," https://github.com/MonetDB/MonetDB/milestone/1?closed=1, 2025, accessed: 2026-03-25.

[26] M. Wang, Z. Wu, X. Xu, J. Liang, C. Zhou, H. Zhang, and Y. Jiang, "Industry practice of coverage-guided enterprise-level dbms fuzzing," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 328–337.

[27] C. Zhang, M. Bai, Y. Zheng, Y. Li, W. Ma, X. Xie, Y. Li, L. Sun, and Y. Liu, "Understanding large language model based fuzz driver generation," *arXiv e-prints*, pp. arXiv–2307, 2023.

[28] Google, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," 2025. [Online]. Available: https://arxiv.org/abs/2507.06261

[29] "Db-engines ranking — popularity ranking of database management systems," https://db-engines.com/en/ranking, accessed: June 25, 2026.

[30] "The database system to speed up your analytical jobs," https://www.monetdb.org/, 1 2024, accessed: June 25, 2026.

[31] "Virtuoso github." https://github.com/openlink/virtuoso-opensource, accessed: June 25, 2026.

[32] "Query billions of rows in milliseconds," https://clickhouse.com/, 1 2024, accessed: June 25, 2026.

[33] S. Bhosale, M. T. Patil, and M. P. Patil, "Sqlite: Light database system," *Int. J. Comput. Sci. Mob. Comput*, vol. 44, no. 4, pp. 882–885, 2015.

[34] "Mysql," https://www.mysql.com/, 1 2024, accessed: June 25, 2026.

[35] "Postgresql," https://www.postgresql.org/, 1 2024, accessed: June 25, 2026.

[36] W. Zhou, Y. Gao, X. Zhou, and G. Li, "Cracking sql barriers: An llm-based dialect translation system," *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–26, 2025.

[37] M. Rigger and Z. Su, "Sqlancer," 2023, accessed: 2026-01-26. [Online]. Available: https://github.com/sqlancer/sqlancer

[38] C. chen, "Squirrel website," https://github.com/s3team/Squirrel, accessed: June 25, 2026.

[39] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 667–682.

[40] J. Wei, P. Chen, K. Lu, J. Dai, and X. Sun, "Sqlaser: Detecting dbms logic bugs with clause-guided fuzzing," *arXiv preprint arXiv:2407.04294*, 2024.

[41] J. Ba and M. Rigger, "Testing database engines via query plan guidance," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2060–2071.

[42] J. Song, W. Dou, Z. Cui, Q. Dai, W. Wang, J. Wei, H. Zhong, and T. Huang, "Testing database systems via differential query execution," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2072–2084.

[43] Z. Cui, W. Dou, Q. Dai, J. Song, W. Wang, J. Wei, and D. Ye, "Differentially testing database transactions for fun and profit," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.

[44] Z. Hao, Q. Huang, C. Wang, J. Wang, Y. Zhang, R. Wu, and C. Zhang, "Pinolo: Detecting logical bugs in database management systems with approximate query synthesis," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 345–358.

[45] X. Gao, Z. Liu, J. Cui, H. Li, H. Zhang, K. Wei, and K. Zhao, "A comprehensive survey on database management system fuzzing: Techniques, taxonomy and experimental comparison," *arXiv preprint arXiv:2311.06728*, 2023.

[46] S. Zhong and M. Rigger, "Testing database systems with large language model synthesized fragments," *arXiv preprint arXiv:2505.02012*, 2025.

[47] L. Lin, Q. Zhu, H. Chen, Z. Wang, R. Wu, and X. Xie, "Qtran: Extending metamorphic-oracle based logical bug detection techniques for multiple-dbms dialect support," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 731–752, 2025.

[48] "Morph artifact and experimental data," Anonymous Zenodo archive, 2026. [Online]. Available: https://doi.org/10.5281/zenodo.19251660